Yes/No Speech Classification: A Reproducible Exercise Solution

1. Objective and learning value

The goal is to build a binary classifier that assigns each spoken word segment to **yes** or **no**. The TorchAudio YESNO corpus used in the supplied exercise contains short, ordered utterances recorded at 8 kHz; each recording is annotated with eight yes/no labels. The supplied implementations describe the speech as Hebrew. This makes the exercise a compact example of a full speech-processing pipeline: audio inspection, word segmentation, feature extraction, model fitting, and evaluation on data that has not leaked from the training recordings. [1] [2] [3]

The central learning point is that a high score is not meaningful unless the word boundaries and labels are credible and the evaluation split matches the intended claim. A classifier evaluated on words from recordings it has already seen may exploit recording-specific patterns. Therefore, all words from one recording must stay together in the same split. The main reference notebook and several other implementations use recording-aware grouping for this reason. [1] [2] [3]

2. Task definition and data

Each audio recording contains a sequence of words and a corresponding sequence of eight binary labels. Treat each segmented word as one example:

- **Input:** waveform samples for one word.
- **Target:** `1` for yes and `0` for no, following the corpus label convention.
- **Group:** the recording ID from which the word came.

Several supplied implementations report excluding recording index 21 because they could not reliably align its detected speech regions with all eight labels. This is a sensible data-quality decision only when it is explicitly recorded and justified; it should not be hidden as if every file were processed successfully. Some solutions also use manual boundary corrections. [1] [2] [3] [4]

3. Processing workflow

Plain Text

```
YESNO recordings + ordered labels
                  ↓
Inspect waveform and detect word regions
                  ↓
Verify region count, boundaries, and label order
                  ↓
MFCC frames → mean and standard deviation per coefficient
                  ↓
Group recordings into train / validation / test
                  ↓
Fit baseline classifiers; tune without using test labels
                  ↓
Report class-wise metrics, confusion matrix, and grouped CV
```

Step 1 — Inspect the recordings

Load the corpus and inspect sample rate, waveform duration, and label sequence. Plot waveform amplitude and short-time energy for representative recordings. The YESNO corpus is small enough that visual checks are practical and valuable. Energy-based segmentation appears across the supplied solutions, but their frame lengths, thresholds, gap rules, and manual corrections differ. A detector returning exactly eight regions is not, by itself, proof that each boundary is correct: a click can be mistaken for a word, or adjacent words can be merged. 1 2 3 4

Step 2 — Segment and verify the words

A useful starting point is short-time root-mean-square (RMS) energy. For frame `m` with `N` samples,

$$\mathrm{RMS}(m) = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} x_m[n]^2}.$$

Use overlapping frames (for example, 20–25 ms with a 10 ms hop), then mark frames whose RMS exceeds an adaptive threshold. Merge very short gaps, remove implausibly short regions, and pad accepted boundaries slightly to preserve initial and final consonants. These are starting parameters, not universal constants. The references use different frame sizes and threshold heuristics, reinforcing the need to inspect detections rather than treat one threshold as ground truth. 1 2 3 4

For every recording, check that the detected segments align one-to-one and in chronological order with the eight labels. Keep a record of boundary edits and skipped

recordings. If a file cannot be aligned confidently, exclude it from the primary analysis and state why. Do not silently force the labels onto uncertain segments.

Step 3 — Extract a compact feature vector

For each word, compute 13 Mel-frequency cepstral coefficients (MFCCs) across its frames. Summarize each coefficient over time using its mean and standard deviation. This yields a 26-dimensional vector:

$$\mathbf{z} = [\mu_1, \ldots, \mu_{13}, \sigma_1, \ldots, \sigma_{13}].$$

This representation captures average spectral shape and some within-word variation while giving every word a fixed-length representation. One reviewed notebook uses 13 MFCCs with 40 mel bands, a 256-point FFT, and an 80-sample hop at 8 kHz. Other supplied work adds MFCC deltas or uses different settings; those are valid variants, but results should not be compared as if features were held constant. 1 2 3 4

Step 4 — Split by recording, not by word

Make a grouped holdout split using recording IDs. A suitable demonstration split is approximately 80% of recordings for training and 20% for the final test, with a fixed random seed. Use grouped cross-validation on the training recordings for model comparison. Fit scaling only on training data, preferably inside a scikit-learn `Pipeline`; otherwise information from validation folds may leak into the feature transformation. 1 2 3

On this small corpus, split proportions can produce somewhat different numbers of words because recording-level grouping is the unit of allocation. Report both the number of recordings and the number of word segments in each partition. Never randomly divide individual segments from the same recording across train and test if the aim is to estimate performance on unseen recordings.

Step 5 — Compare baseline classifiers

Compare a fixed set of models using the same segments, features, and splits. A useful minimum is an RBF-kernel support vector machine (SVM), k-nearest neighbors (k-NN), Random Forest, and a small multilayer perceptron (MLP). Include Logistic Regression if an additional interpretable baseline is desired. Standardize features for SVM, k-NN, MLP, and Logistic Regression; tree models generally do not require scaling. Choose hyperparameters using training folds or validation data only. The supplied projects include these model families, though their exact parameter values differ. 1 2 3 4

4. Illustrative implementation pattern

The following code shows the key feature and evaluation structure. It assumes that verified word waveforms, binary labels, and recording IDs have already been assembled into `segments`, `y`, and `groups`. Boundary detection is deliberately kept as a separately validated stage because the supplied implementations use distinct heuristics and some require manual review.

Python

```python
import numpy as np
import torch
import torchaudio
from sklearn.model_selection import GroupShuffleSplit, StratifiedGroupKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    f1_score
)

SR = 8000
mfcc_transform = torchaudio.transforms.MFCC(
    sample_rate=SR,
    n_mfcc=13,
    melkwargs={"n_fft": 256, "hop_length": 80, "n_mels": 40},
)

def word_features(waveform):
    """waveform: mono float tensor shaped [1, samples]."""
    if waveform.ndim == 1:
        waveform = waveform.unsqueeze(0)
    coeff = mfcc_transform(waveform)       # [1, 13, frames]
    mean = coeff.mean(dim=-1).squeeze(0)
    std = coeff.std(dim=-1, unbiased=False).squeeze(0)
    return torch.cat([mean, std]).cpu().numpy()

X = np.vstack([word_features(w) for w in segments])
y = np.asarray(y, dtype=int)
groups = np.asarray(groups)  # recording ID, repeated for its word segments

# Hold out whole recordings for the final evaluation.
holdout = GroupShuffleSplit(n_splits=1, test_size=0.20, random_state=42)
train_idx, test_idx = next(holdout.split(X, y, groups))
X_train, X_test = X[train_idx], X[test_idx]
```

```
y_train, y_test = y[train_idx], y[test_idx]
g_train = groups[train_idx]

models = {
    "SVM-RBF": make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1.0)),
    "k-NN": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
    "Random Forest": RandomForestClassifier(
        n_estimators=200, random_state=42, class_weight="balanced"
    ),
    "MLP": make_pipeline(
        StandardScaler(),
        MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=1000,
                      early_stopping=True, random_state=42)
    ),
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(f"\\n{name}")
    print("Accuracy:", accuracy_score(y_test, pred))
    print("Macro-F1:", f1_score(y_test, pred, average="macro"))
    print("Confusion matrix [true rows, predicted columns]:\\n",
          confusion_matrix(y_test, pred, labels=[0, 1]))
    print(classification_report(
        y_test, pred, labels=[0, 1], target_names=["no", "yes"],
        zero_division=0
    ))

# Grouped cross-validation example for the SVM on training recordings.
cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
svm = models["SVM-RBF"]
fold_scores = []
for fit_idx, val_idx in cv.split(X_train, y_train, g_train):
    svm.fit(X_train[fit_idx], y_train[fit_idx])
    val_pred = svm.predict(X_train[val_idx])
    fold_scores.append(f1_score(y_train[val_idx], val_pred, average="macro"))
print("SVM grouped CV macro-F1 mean/std:",
      np.mean(fold_scores), np.std(fold_scores, ddof=1))
```

Important implementation note: the code above reports a single held-out test result for each listed baseline. If hyperparameters are tuned, tune them in grouped folds or on a distinct validation partition; do not repeatedly choose settings based on the test set. When using a grouped CV loop, the scaler must be fitted separately inside each fold. The `Pipeline` in the example ensures this for the SVM. For a more complete comparison, run each

candidate through the same grouped folds and summarize fold-level accuracy and macro-F1.

5. Evaluation and interpretation

Report at least accuracy, precision, recall, and F1 for each class, macro-F1, and a confusion matrix. Accuracy is the fraction of correct predictions, while macro-F1 gives equal weight to the two classes by averaging their class-wise F1 scores. The confusion matrix shows which word is being confused with which. Also report the number of segments and recordings in each split, the grouped cross-validation mean and variability, and whether test labels were used only once for final evaluation.

The supplied notebooks and reports contain very high stored scores, including perfect results for some model/split combinations, but these are outputs from particular implementations and are not fresh reproductions here. One reference reports perfect held-out and grouped-fold scores for several models; another reports differing held-out and noise-test results. These values are dataset- and setup-specific, not evidence that the task is universally error-free. Differences in segmentation, features, split design, model settings, and even data corpus prevent treating them as a single leaderboard. 1 2 3 4

A practical error analysis is to list false positives and false negatives alongside their recording IDs, segment durations, and boundary plots. If errors cluster around short or noisy segments, inspect segmentation first. If errors remain despite correct boundaries, compare acoustic features or model behavior. Avoid causal claims about why a classifier succeeds unless the experiment was designed to test that explanation.

6. Optional extension: robustness to noise

To study robustness, add controlled white noise to test waveforms at specified signal-to-noise ratios (SNRs), while retaining a clean-trained model. State how noise power is set, list SNR levels, fix random seeds, and repeat several noise draws per level. Report mean and variability at each SNR. The supplied work includes noise experiments, but protocols and reported detail differ; a small synthetic-noise test does not establish performance under real-world noise, new speakers, or different microphones. 2 4

7. Conclusion

This exercise demonstrates that speech classification is not only a classifier-selection problem. Reliable results depend on valid segmentation, correct label alignment, consistent feature extraction, and an evaluation split that prevents recording-level leakage. A compact MFCC-statistics representation and a grouped comparison of SVM, k-NN, Random Forest, and MLP provide an accessible baseline. The practical value is a repeatable

analysis in which learners can explain what each stage contributes, interpret mistakes, and make appropriately limited claims about what the measured scores show.

Confidence in the shared exercise structure: High. The core pipeline and dataset-level task are consistent across multiple supplied references. **Confidence in any individual reported score as a reproducible benchmark: Low to Medium.** The repositories contain stored outputs and README claims, but these were not rerun as part of this solution, and the experimental setups differ.

References

[1] Reviewed TorchAudio YESNO classification notebook

[2] Digital speech processing YESNO classification notebook

[3] SpeechProcessingCourse Mini-Task 8 report

[4] YESNO classification README update and referenced implementation

[5] Speech Commands yes/no classification notebook